UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

CS2032 - Cloud Computing (Ciclo 2026-2)
Multi-tenancy
Semana 9 - Taller 2: Microservicio Multi-tenancy

ELABORADO POR: GERALDO COLCHADO

Contenido

Microservicio Multi-tenancy

1. **Objetivo del taller 2**
2. Ejercicio 1: Crear una función Lambda - ListarAlumnos
3. Ejercicio 2: Crear una función Lambda - CrearAlumno
4. Ejercicio 3: Crear una función Lambda - ModificarAlumno
5. Ejercicio 4: Crear una función Lambda - BuscarAlumno
6. Ejercicio 5: Crear una función Lambda - EliminarAlumno
7. Ejercicio 6: Publicar microservicio como API Rest CRUD con servicio Api Gateway
8. Ejercicio 7: Ejercicio propuesto
9. Cierre

Objetivo del taller 2:
Microservicio Multi-tenancy

- Diseñar e implementar un Microservicio Multi-tenancy con servicio Lambda (Serverless) y Api Gateway (Serverless)

Contenido

Microservicio Multi-tenancy

1. Objetivo del taller 2
2. **Ejercicio 1: Crear una función Lambda - ListarAlumnos**
3. Ejercicio 2: Crear una función Lambda - CrearAlumno
4. Ejercicio 3: Crear una función Lambda - ModificarAlumno
5. Ejercicio 4: Crear una función Lambda - BuscarAlumno
6. Ejercicio 5: Crear una función Lambda - EliminarAlumno
7. Ejercicio 6: Publicar microservicio como API Rest CRUD con servicio Api Gateway
8. Ejercicio 7: Ejercicio propuesto
9. Cierre

Function as a Service (FaaS)
Lambda (Serverless)

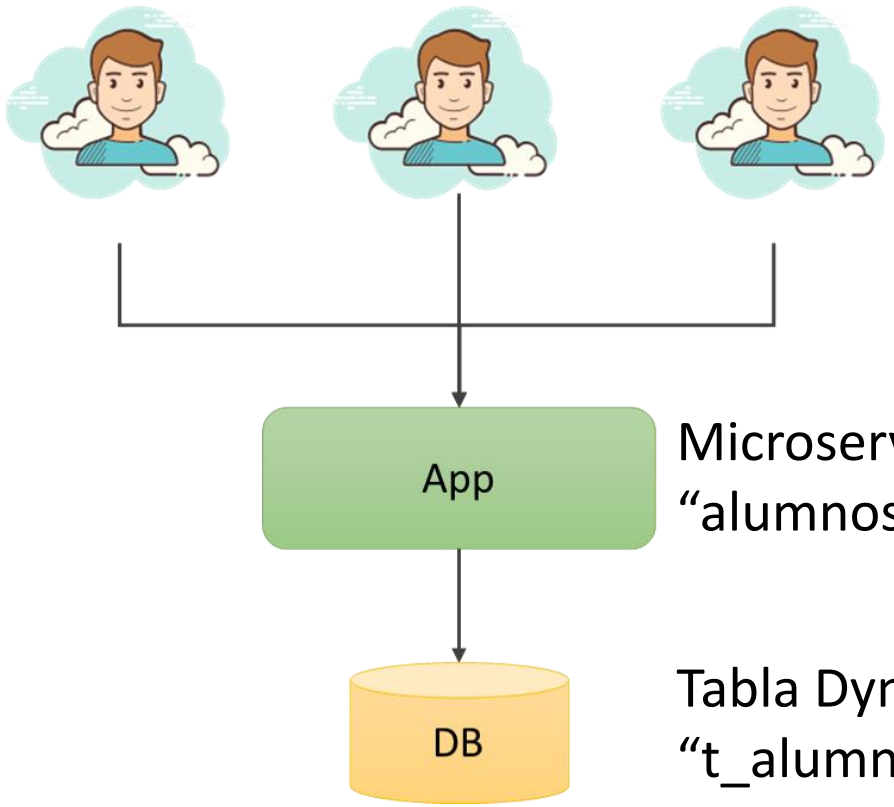


Funcionamiento

AWS Lambda es un servicio informático sin servidor y basado en eventos que le permite ejecutar código para prácticamente cualquier tipo de aplicación o servicio backend sin necesidad de aprovisionar o administrar servidores. Puede activar Lambda desde más de 200 servicios de AWS y aplicaciones de software como servicio (SaaS), y solo paga por lo que utiliza.

Fuente: https://aws.amazon.com/es/lambda/

Patrones de diseño BD Multi-tenancy
Patrón 1: En DynamoDB (NoSQL)



Microservicio
“alumnos”

Tabla DynamoDB
“t_alumnos”

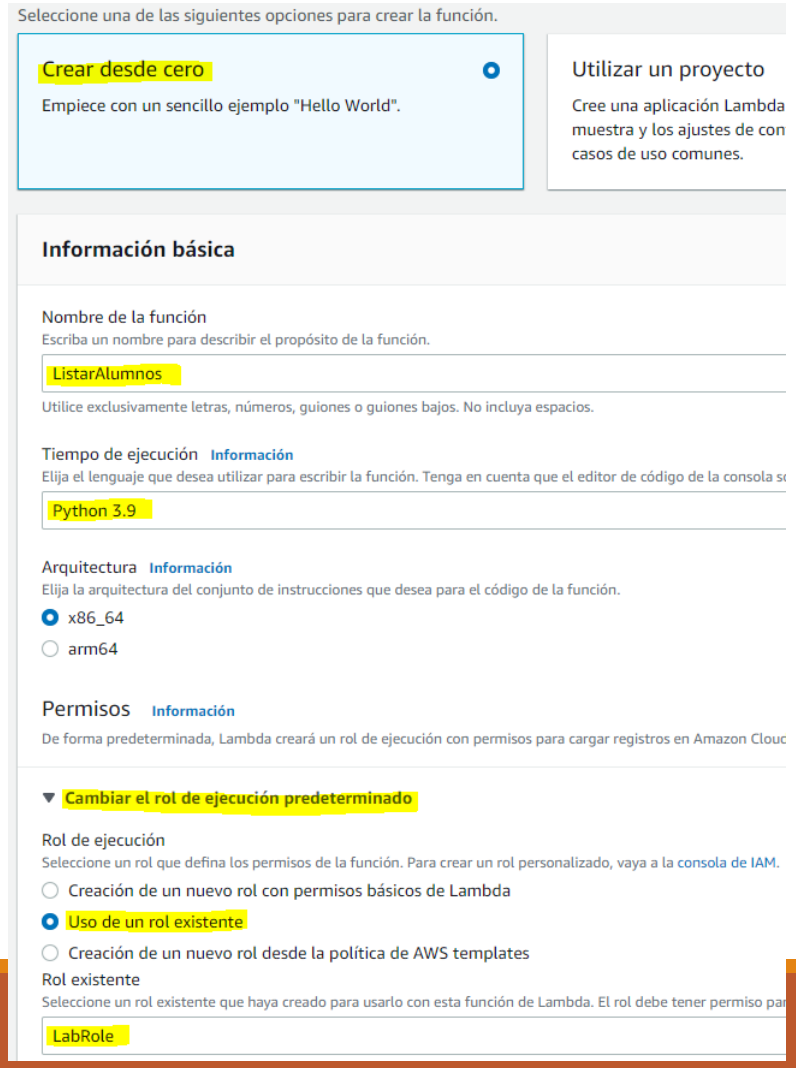
Ejercicio: Diseñe e implemente un microservicio Multi-tenancy para almacenar los alumnos de una universidad. Considerar que el tenant_id es un código por cada universidad en mayúsculas:

tenant_id: UTEC, UNIV2, UNIV3, UNIV4, etc.

Ejercicio 1:
Crear una función Lambda - ListarAlumnos

- **Paso 1:** Requisito: Tener creada la tabla DynamoDB t_alumnos
- **Paso 2:** Crear la función lambda

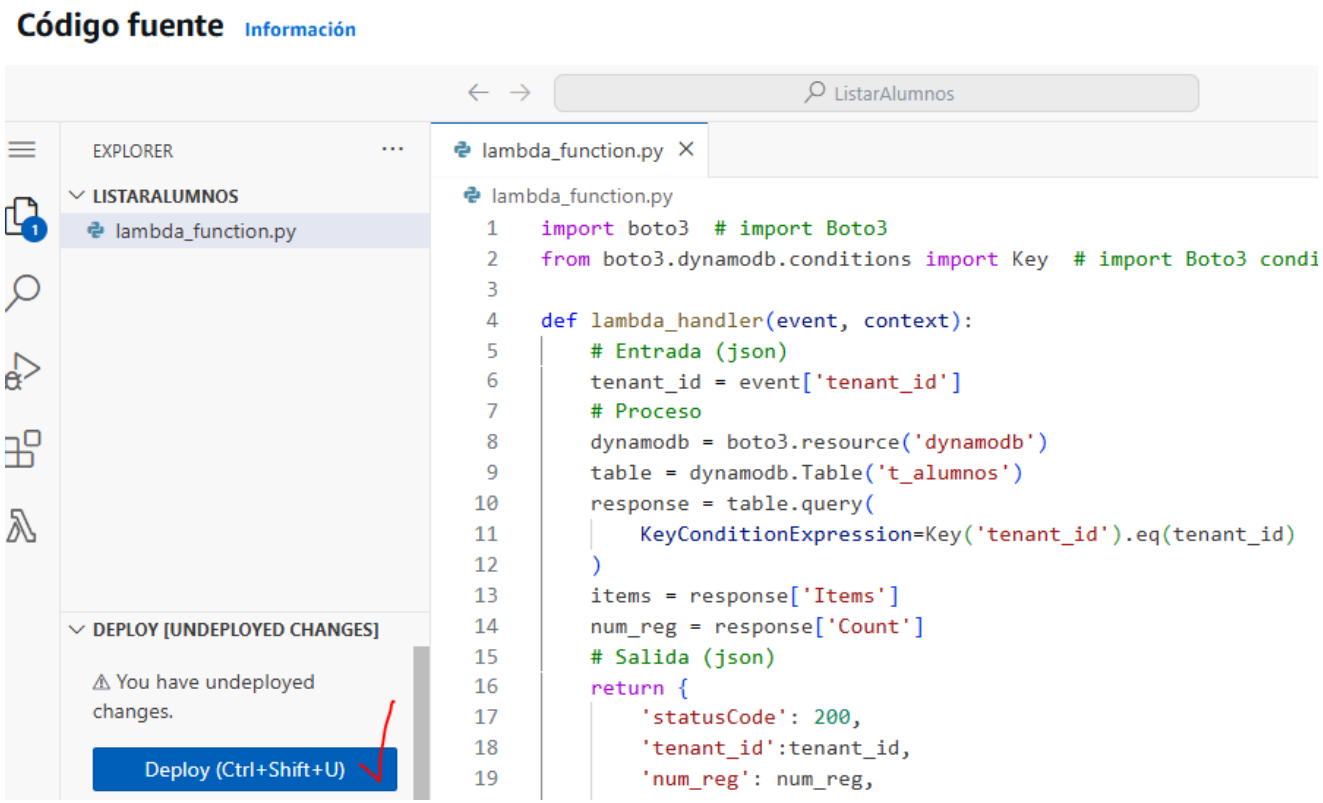
Crear una función



Crear una función

Ejercicio 1:
Crear una función Lambda - ListarAlumnos

- **Paso 3:** Escribir el siguiente código fuente y “Deploy”



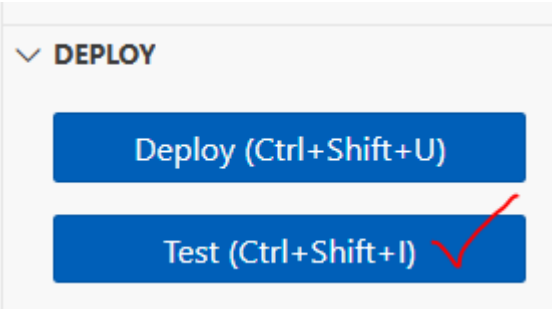

```
import boto3  # import Boto3
from boto3.dynamodb.conditions import Key  # import Boto3 conditions

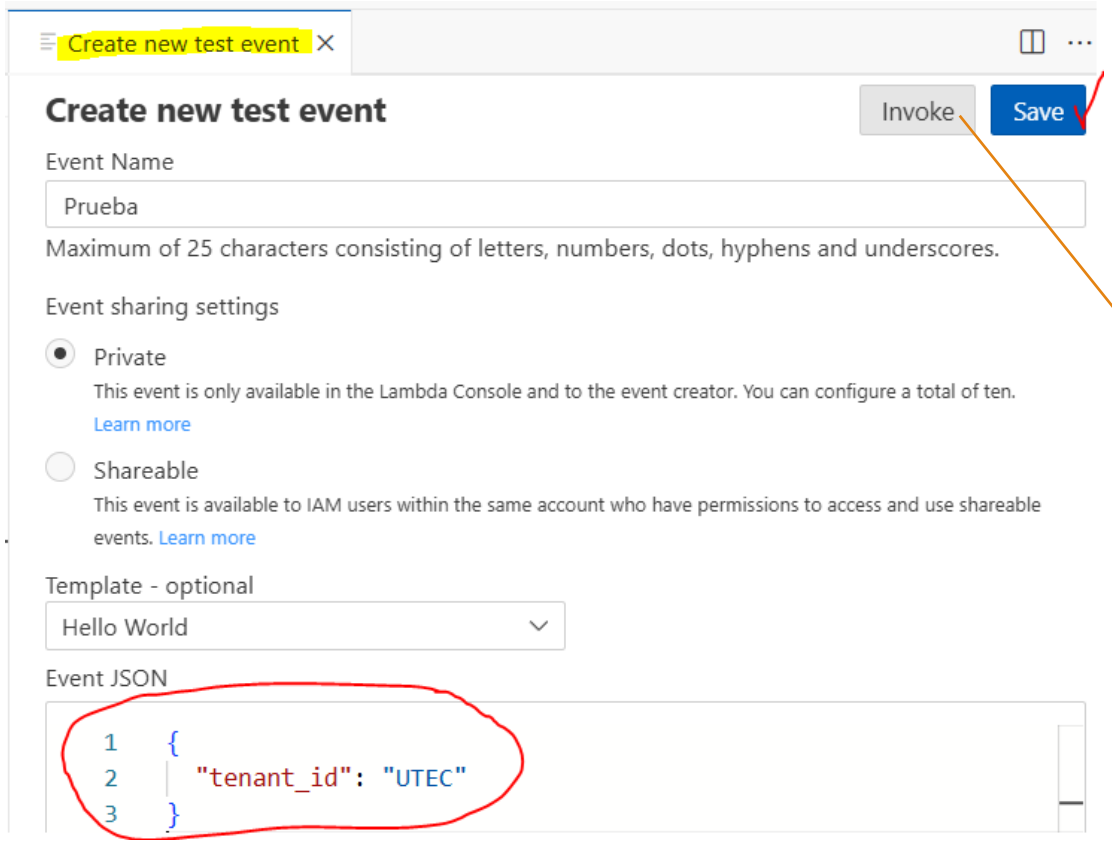
def lambda_handler(event, context):
    # Entrada (json)
    tenant_id = event['tenant_id']
    # Proceso
    dynamodb = boto3.resource('dynamodb')
    table = dynamodb.Table('t_alumnos')
    response = table.query(
        KeyConditionExpression=Key('tenant_id').eq(tenant_id)
    )
    items = response['Items']
    num_reg = response['Count']
    # Salida (json)
    return {
        'statusCode': 200,
        'tenant_id':tenant_id,
        'num_reg': num_reg,
        'alumnos': items
    }
```

Ejercicio 1:
Crear una función Lambda - ListarAlumnos

- **Paso 4:** Probar el lambda con esta entrada

```
{
  "tenant_id": "UTEC"
}
```







Contenido

Microservicio Multi-tenancy

1. Objetivo del taller 2
2. Ejercicio 1: Crear una función Lambda - ListarAlumnos
3. **Ejercicio 2: Crear una función Lambda - CrearAlumno**
4. Ejercicio 3: Crear una función Lambda - ModificarAlumno
5. Ejercicio 4: Crear una función Lambda - BuscarAlumno
6. Ejercicio 5: Crear una función Lambda - EliminarAlumno
7. Ejercicio 6: Publicar microservicio como API Rest CRUD con servicio Api Gateway
8. Ejercicio 7: Ejercicio propuesto
9. Cierre

Ejercicio 2:
Crear una función Lambda - CrearAlumno

- **Paso 1:** Escribir el siguiente código fuente y “Deploy”

- **Paso 2:** Probar el lambda con esta entrada:

```
{
  "tenant_id": "UTEC",
  "alumno_id": "202099998",
  "alumno_datos": {
            "nombre": "Geraldo Colchado",
            "sexo": "M",
            "fecha_nac": "2000-09-01",
            "celular": "999736332",
            "domicilio": {
                        "direcc": "Av. Javier Prado 158",
                        "distrito": "San Isidro",
                        "provincia": "Lima",
                        "departamento": "Lima",
                        "pais": "Perú"
            }
  }
}
```

```
import boto3

def lambda_handler(event, context):
    # Entrada (json)
    tenant_id = event['tenant_id']
    alumno_id = event['alumno_id']
    alumno_datos = event['alumno_datos']
    # Proceso
    dynamodb = boto3.resource('dynamodb')
    table = dynamodb.Table('t_alumnos')
    alumno = {
        'tenant_id': tenant_id,
        'alumno_id': alumno_id,
        'alumno_datos': alumno_datos
    }
    response = table.put_item(Item=alumno)
    # Salida (json)
    return {
        'statusCode': 200,
        'response': response
    }
```

Contenido

Microservicio Multi-tenancy

1. Objetivo del taller 2
2. Ejercicio 1: Crear una función Lambda - ListarAlumnos
3. Ejercicio 2: Crear una función Lambda - CrearAlumno
4. **Ejercicio 3: Crear una función Lambda - ModificarAlumno**
5. Ejercicio 4: Crear una función Lambda - BuscarAlumno
6. Ejercicio 5: Crear una función Lambda - EliminarAlumno
7. Ejercicio 6: Publicar microservicio como API Rest CRUD con servicio Api Gateway
8. Ejercicio 7: Ejercicio propuesto
9. Cierre

Ejercicio 3:
Crear una función Lambda - ModificarAlumno

- **Paso 1:** Escribir el siguiente código fuente y “Deploy”

- **Paso 2:** Probar el lambda con esta entrada:

```
{
  "tenant_id": "UTEC",
  "alumno_id": "202099998",
  "alumno_datos": {
            "nombre": "Geraldo Colchado Ruiz",
            "sexo": "M",
            "fecha_nac": "2000-09-01",
            "celular": "999736332",
            "domicilio": {
                        "direcc": "Av. Javier Prado 158",
                        "distrito": "San Borja",
                        "provincia": "Lima",
                        "departamento": "Lima",
                        "pais": "Perú"
            }
  }
}
```

```
import boto3

def lambda_handler(event, context):
    # Entrada (json)
    tenant_id = event['tenant_id']
    alumno_id = event['alumno_id']
    alumno_datos = event['alumno_datos']
    # Proceso
    dynamodb = boto3.resource('dynamodb')
    table = dynamodb.Table('t_alumnos')
    response = table.update_item(
        Key={
            'tenant_id': tenant_id,
            'alumno_id': alumno_id
        },
        UpdateExpression="set alumno_datos=:alumno_datos",
        ExpressionAttributeValues={
            ':alumno_datos': alumno_datos
        },
        ReturnValues="UPDATED_NEW"
    )
    # Salida (json)
    return {
        'statusCode': 200,
        'response': response
    }
```

Contenido

Microservicio Multi-tenancy

1. Objetivo del taller 2
2. Ejercicio 1: Crear una función Lambda - ListarAlumnos
3. Ejercicio 2: Crear una función Lambda - CrearAlumno
4. Ejercicio 3: Crear una función Lambda - ModificarAlumno
5. **Ejercicio 4: Crear una función Lambda - BuscarAlumno**
6. Ejercicio 5: Crear una función Lambda - EliminarAlumno
7. Ejercicio 6: Publicar microservicio como API Rest CRUD con servicio Api Gateway
8. Ejercicio 7: Ejercicio propuesto
9. Cierre

Ejercicio 4:
Crear una función Lambda - BuscarAlumno

- **Paso 1:** Escribir el siguiente código fuente y “Deploy”
- **Paso 2:** Probar el lambda con esta entrada:

```
{
  "tenant_id": "UTEC",
  "alumno_id": "202099998"
}
```

```
import boto3

def lambda_handler(event, context):
    # Entrada (json)
    tenant_id = event['tenant_id']
    alumno_id = event['alumno_id']
    # Proceso
    dynamodb = boto3.resource('dynamodb')
    table = dynamodb.Table('t_alumnos')
    response = table.get_item(
        Key={
            'tenant_id': tenant_id,
            'alumno_id': alumno_id
        }
    )
    # Salida (json)
    return {
        'statusCode': 200,
        'response': response
    }
```

Contenido

Microservicio Multi-tenancy

1. Objetivo del taller 2
2. Ejercicio 1: Crear una función Lambda - ListarAlumnos
3. Ejercicio 2: Crear una función Lambda - CrearAlumno
4. Ejercicio 3: Crear una función Lambda - ModificarAlumno
5. Ejercicio 4: Crear una función Lambda - BuscarAlumno
6. **Ejercicio 5: Crear una función Lambda - EliminarAlumno**
7. Ejercicio 6: Publicar microservicio como API Rest CRUD con servicio Api Gateway
8. Ejercicio 7: Ejercicio propuesto
9. Cierre

Ejercicio 5:
Crear una función Lambda - EliminarAlumno

- **Paso 1:** Escribir el siguiente código fuente y “Deploy”
- **Paso 2:** Probar el lambda con esta entrada:

```
{
  "tenant_id": "UTEC",
  "alumno_id": "202099998"
}
```

```
import boto3

def lambda_handler(event, context):
    # Entrada (json)
    tenant_id = event['tenant_id']
    alumno_id = event['alumno_id']
    # Proceso
    dynamodb = boto3.resource('dynamodb')
    table = dynamodb.Table('t_alumnos')
    response = table.delete_item(
        Key={
            'tenant_id': tenant_id,
            'alumno_id': alumno_id
        }
    )
    # Salida (json)
    return {
        'statusCode': 200,
        'response': response
    }
```

Contenido

Microservicio Multi-tenancy

1. Objetivo del taller 2
2. Ejercicio 1: Crear una función Lambda - ListarAlumnos
3. Ejercicio 2: Crear una función Lambda - CrearAlumno
4. Ejercicio 3: Crear una función Lambda - ModificarAlumno
5. Ejercicio 4: Crear una función Lambda - BuscarAlumno
6. Ejercicio 5: Crear una función Lambda - EliminarAlumno
7. **Ejercicio 6: Publicar microservicio como API Rest CRUD con servicio Api Gateway**
8. Ejercicio 7: Ejercicio propuesto
9. Cierre

Ejercicio 6:
Servicio Api Gateway (Serverless)



Amazon API Gateway es un servicio completamente administrado que facilita a los desarrolladores la creación, la publicación, el mantenimiento, el monitoreo y la protección de API a cualquier escala. Las API actúan como la "puerta de entrada" para que las aplicaciones accedan a los datos, la lógica empresarial o la funcionalidad de sus servicios de backend. Con API Gateway, puede crear API RESTful y API WebSocket que permiten aplicaciones de comunicación bidireccional en tiempo real. API Gateway admite cargas de trabajo en contenedores y sin servidor, así como aplicaciones web.

API Gateway gestiona todas las tareas implicadas en la aceptación y el procesamiento de hasta cientos de miles de llamadas a API simultáneas, entre ellas, la administración del tráfico, compatibilidad con CORS, el control de autorizaciones y acceso, la limitación controlada, el monitoreo y la administración de versiones de API. API Gateway no requiere pagos mínimos ni costos iniciales. Se paga por las llamadas a las API que se reciben y por la cantidad de datos salientes transferidos, además, con el modelo de precios por niveles de API Gateway, puede reducir sus costos a medida que cambie la escala de uso de las API.

1 millón de llamadas al API recibidas de forma gratuita

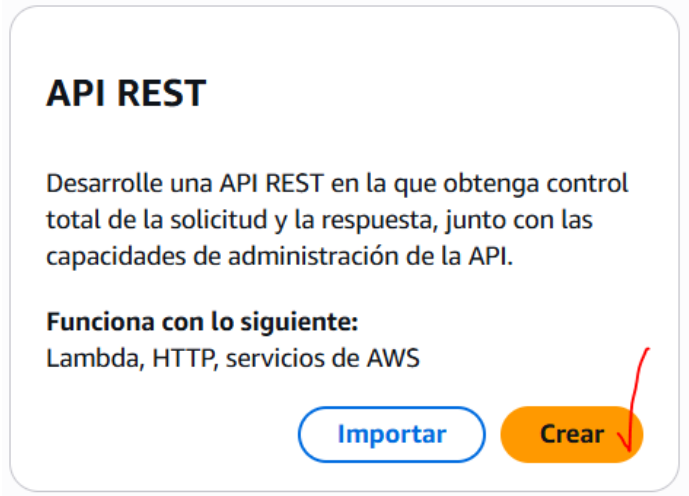
por mes, durante 12 meses con la capa gratuita de AWS

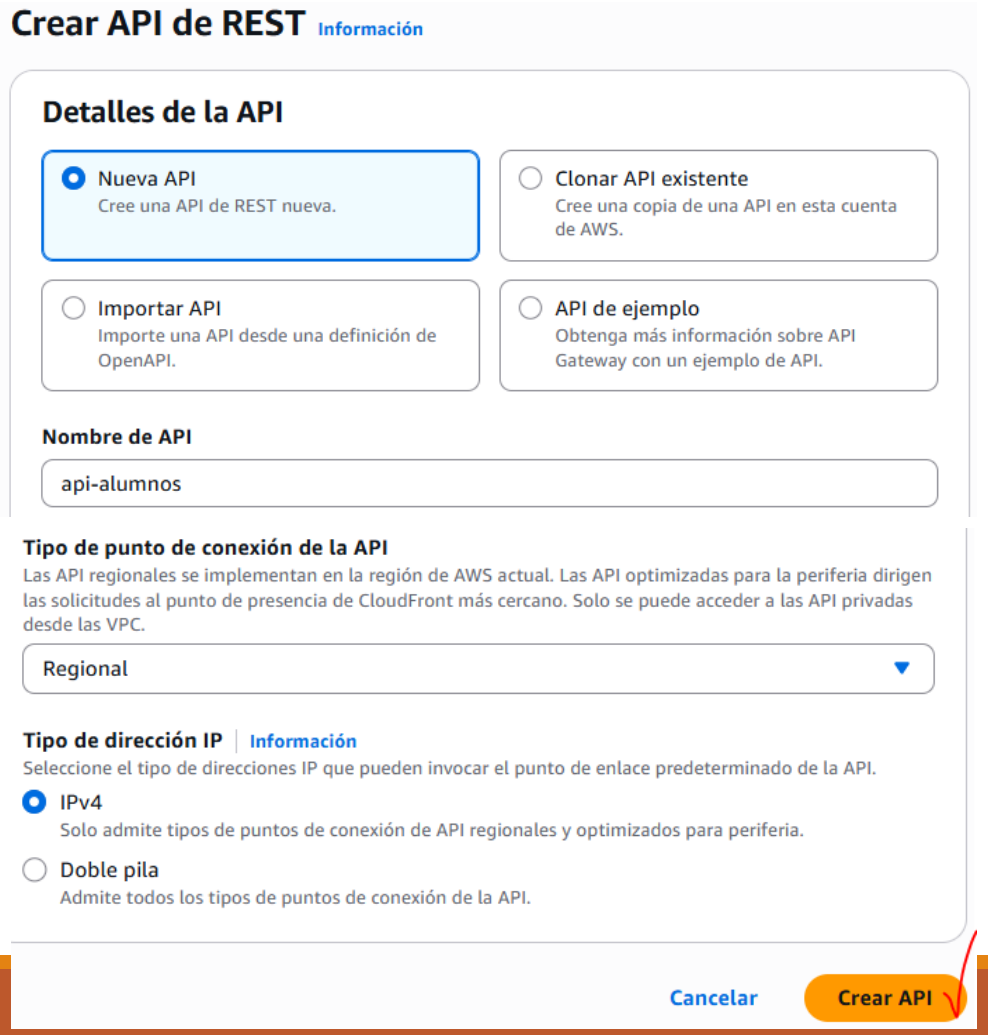
Comience de forma gratuita »

Fuente: https://aws.amazon.com/es/api-gateway/

Ejercicio 6:
Publicar microservicio como API Rest CRUD con servicio Api Gateway

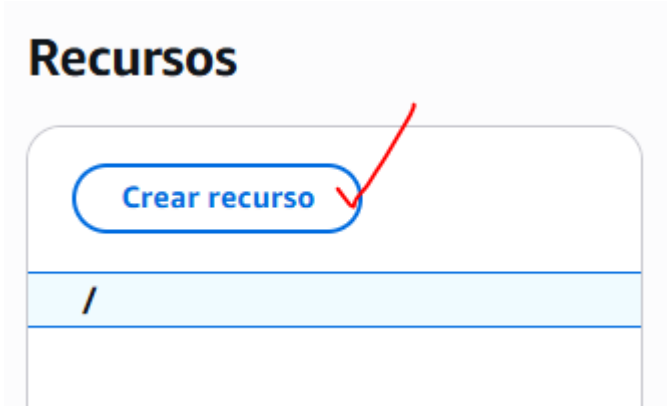
- **Paso 1:** Crear API Rest “api-alumnos” en servicio Api Gateway





Ejercicio 6:
Publicar microservicio como API Rest CRUD con servicio Api Gateway

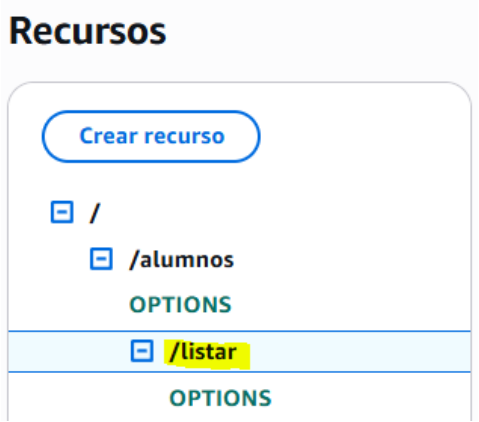
- **Paso 2:** Crear recurso alumnos



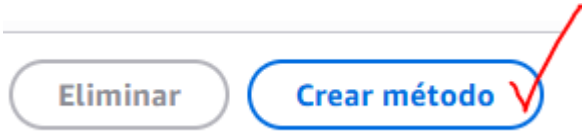


Ejercicio 6:
Publicar microservicio como API Rest CRUD con servicio Api Gateway

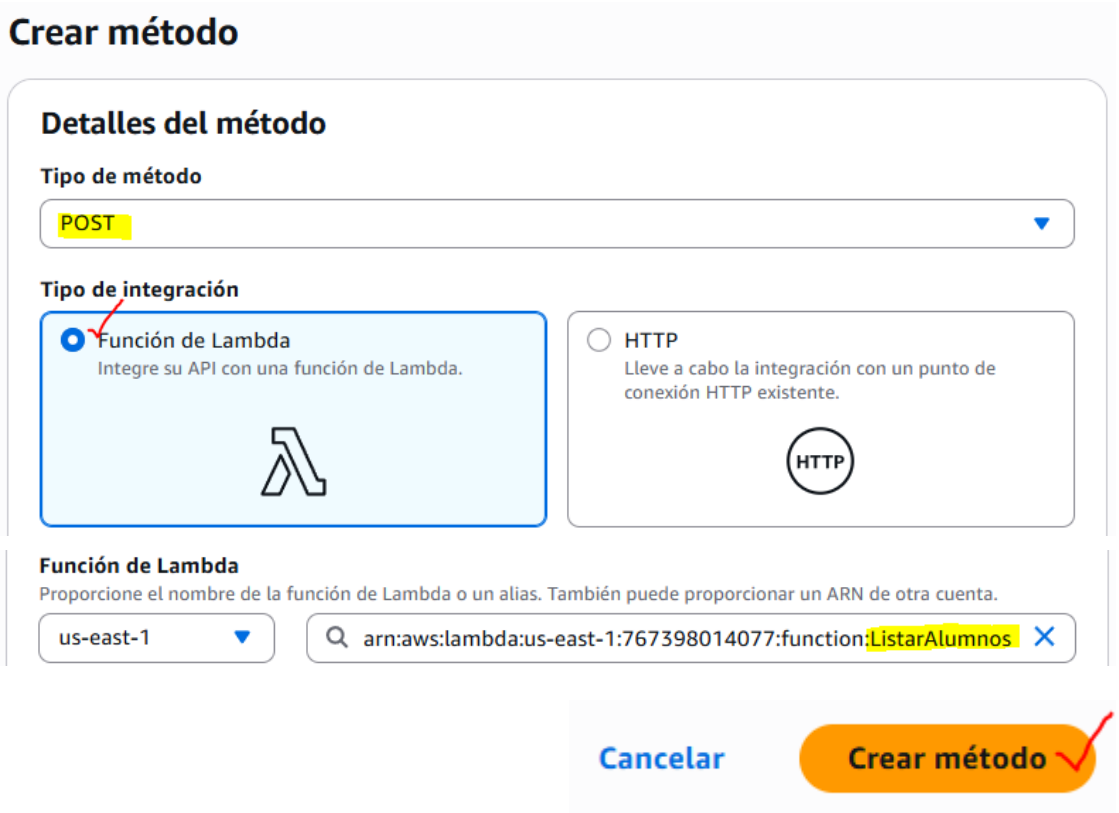
- **Paso 3:** Crear recurso listar



- **Paso 4:** Crear método POST



- **Paso 5:** Asociar a Lambda ListarAlumnos



Ejercicio 6:
Publicar microservicio como API Rest CRUD con servicio Api Gateway

- **Paso 6:** Probar



Cuerpo de la solicitud

```
{
    "tenant_id": "UTEC"
}
```

Pruebas

/alumnos/listar - Resultados de la prueba del método POST

Solicitud
/alumnos/listar

Latencia en ms
3099

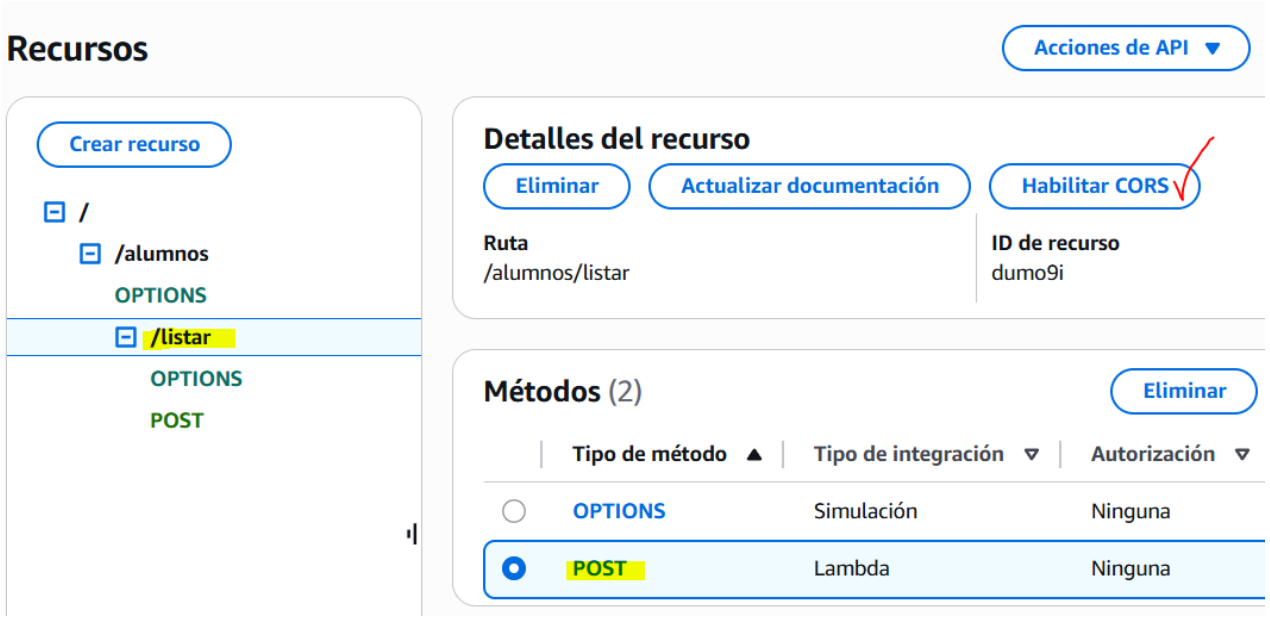
Estado
200

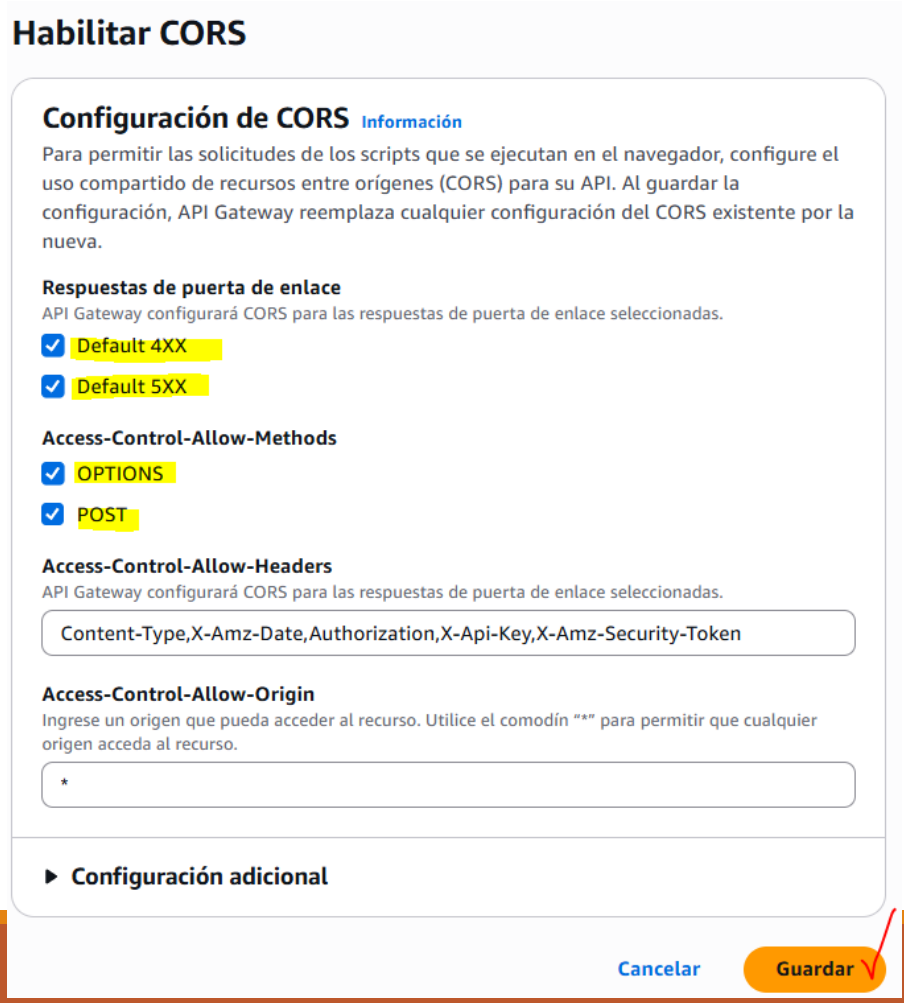
Cuerpo de respuesta


```
{"statusCode": 200, "tenant_id": "UTEC", "num_reg": 3,
"alumnos": [{"alumno_id": "202010290", "tenant_id": "UTEC",
"alumno_datos": {"fecha_nac": "2004-12-04", "celular":
"999736371", "domicilio": {"direcc": "Av. El Polo 1767",
"distrito": "Monterrico", "departamento": "Lima", "provincia":
"Lima", "pais": "Perú"}, "sexo": "M", "nombre": "Juan
Pérez"}}, {"alumno_id": "202012345", "tenant_id": "UTEC",
"alumno_datos": {"fecha_nac": "2005-02-03", "celular":
"999736741", "domicilio": {"direcc": "Av. Javier Prado 1928",
"distrito": "San Borja", "departamento": "Lima", "provincia":
"Lima", "pais": "Perú"}, "sexo": "F", "nombre": "Valeria
Molina"}}, {"alumno_id": "202035215", "tenant_id": "UTEC",
"alumno_datos": {"fecha_nac": "2004-09-01", "celular":
"999736123", "domicilio": {"direcc": "Av. Arequipa 1028",
"distrito": "Lince", "departamento": "Lima", "provincia":
"Lima", "pais": "Perú"}, "sexo": "M", "nombre": "Carlos
Carrasco"}}]}
```

Ejercicio 6:
Publicar microservicio como API Rest CRUD con servicio Api Gateway

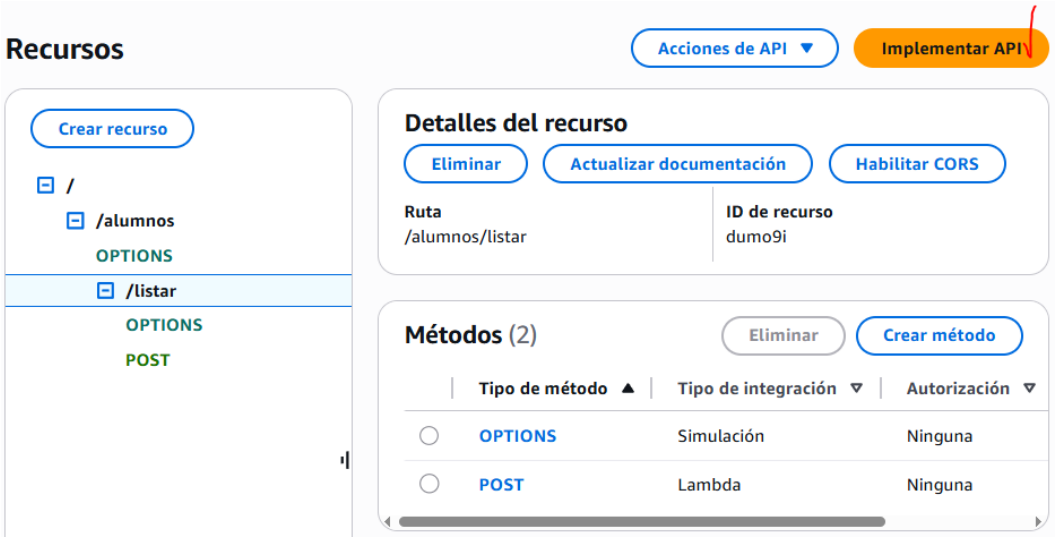
- **Paso 7:** Habilitar CORS

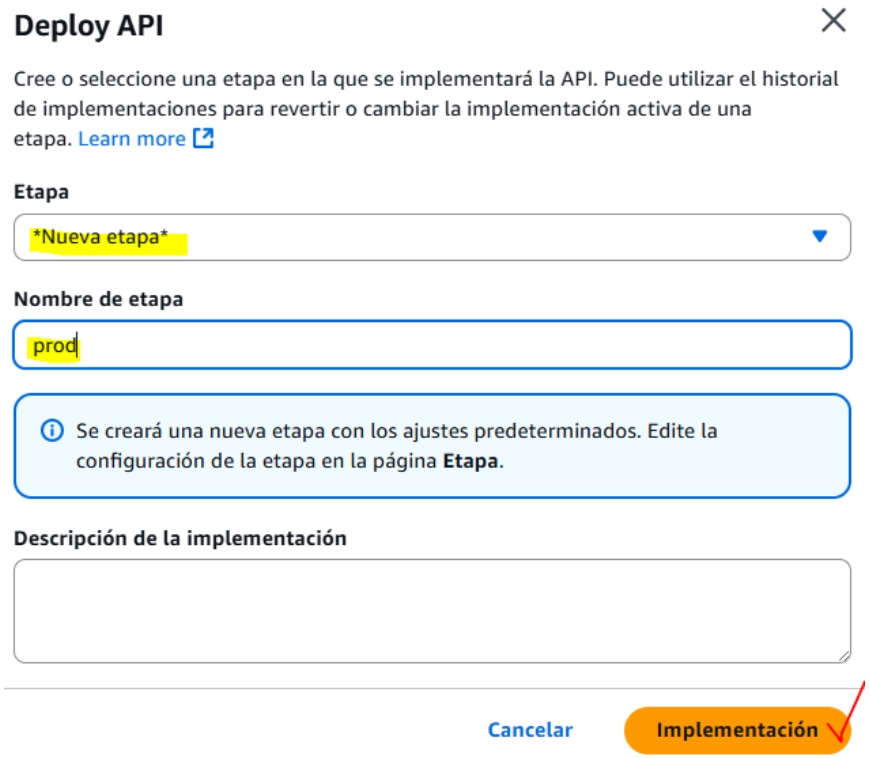




Ejercicio 6:
Publicar microservicio como API Rest CRUD con servicio Api Gateway

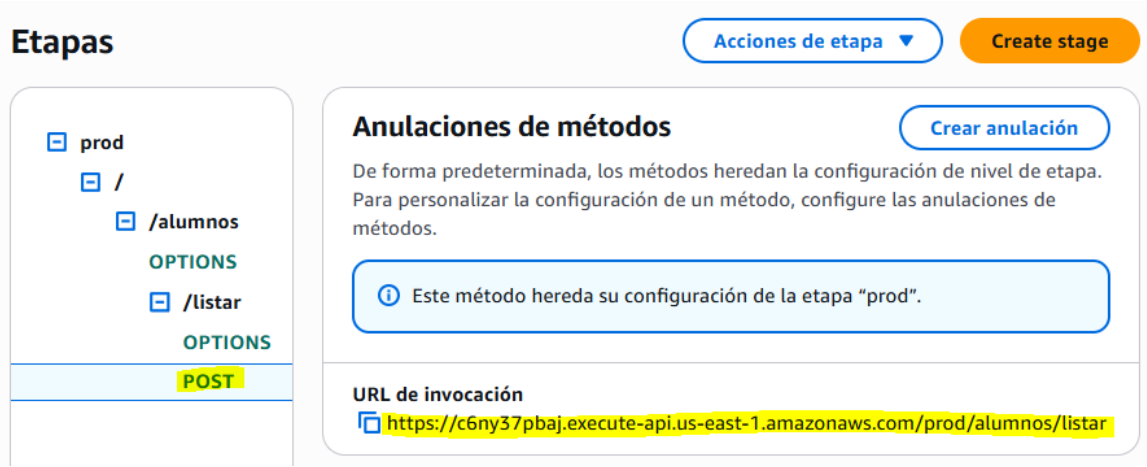
- **Paso 8:** Implementar la Api

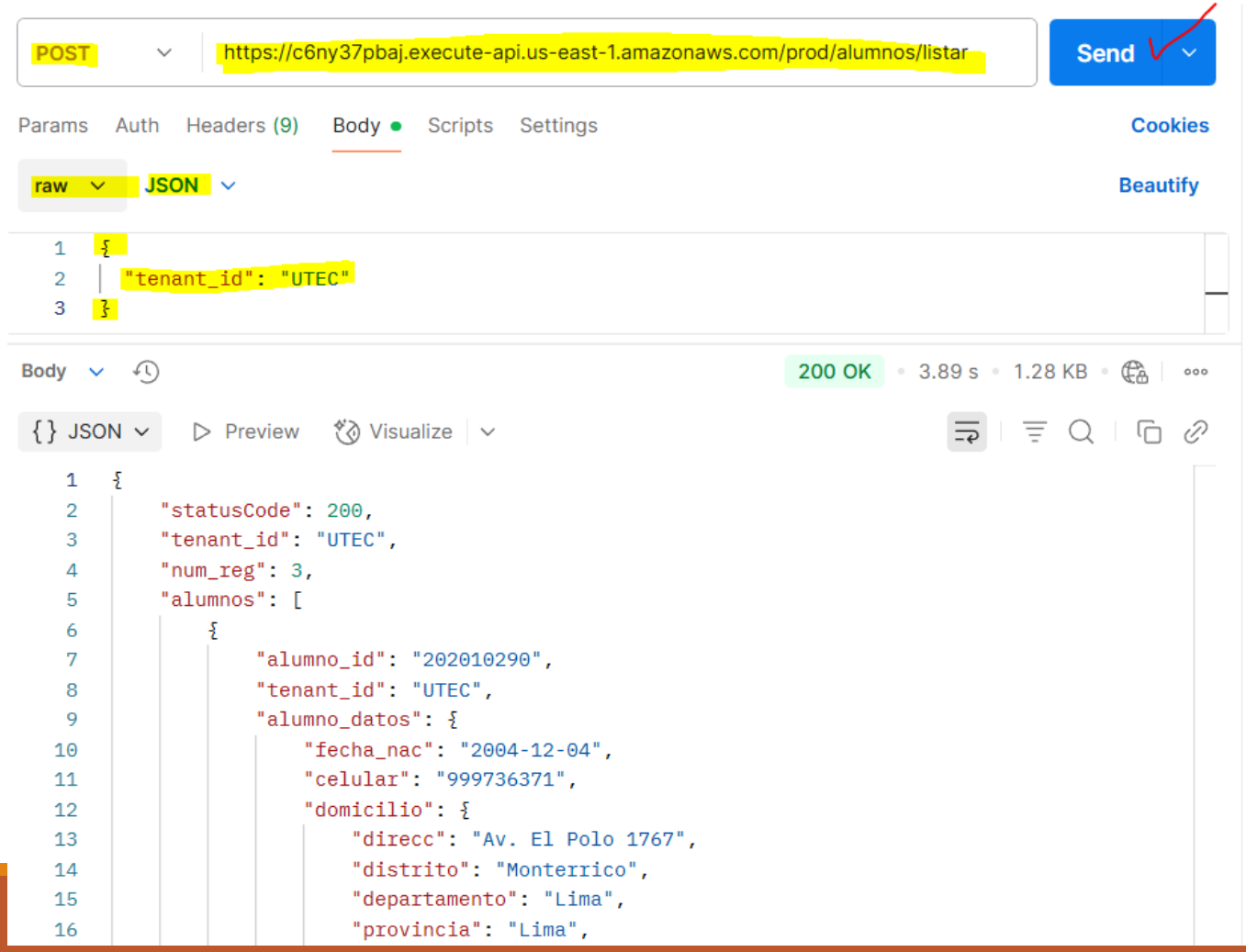




Ejercicio 6:
Publicar microservicio como API Rest CRUD con servicio Api Gateway

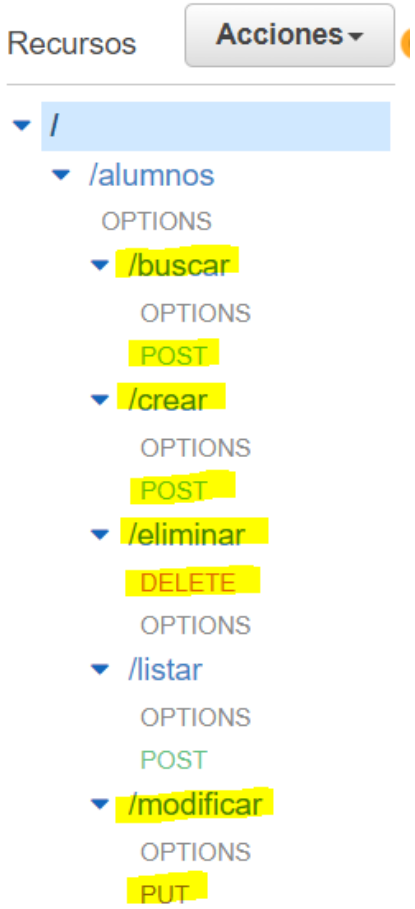
- **Paso 9:** Obtenga el enlace para probar el api y pruebe en postman





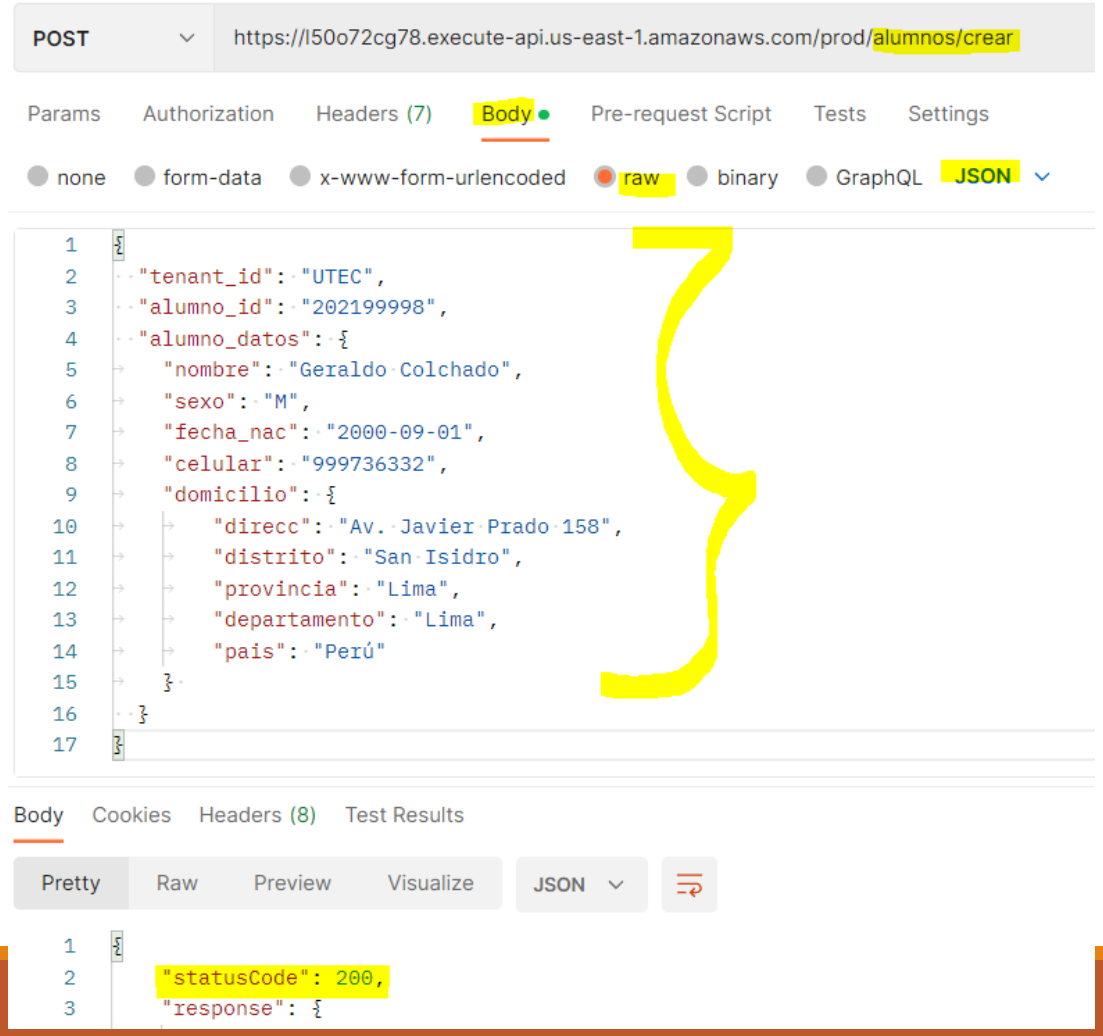
Ejercicio 6:
Publicar microservicio como API Rest CRUD con servicio Api Gateway

- **Paso 10:** Repetir Pasos 3 al 8 para el resto de recursos según esta imagen



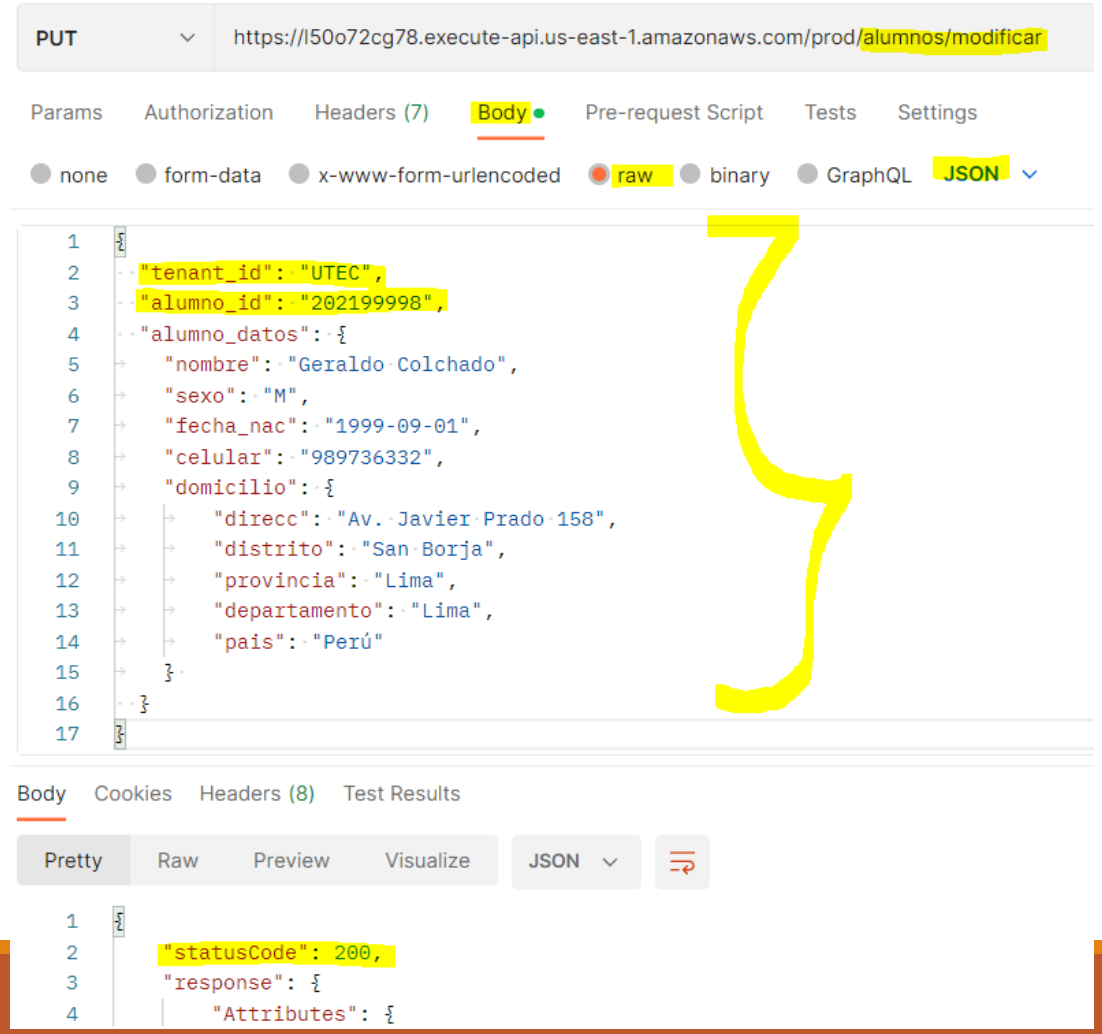
Ejercicio 6:
Publicar microservicio como API Rest CRUD con servicio Api Gateway

- **Paso 11:** Probar la api CrearAlumno en postman



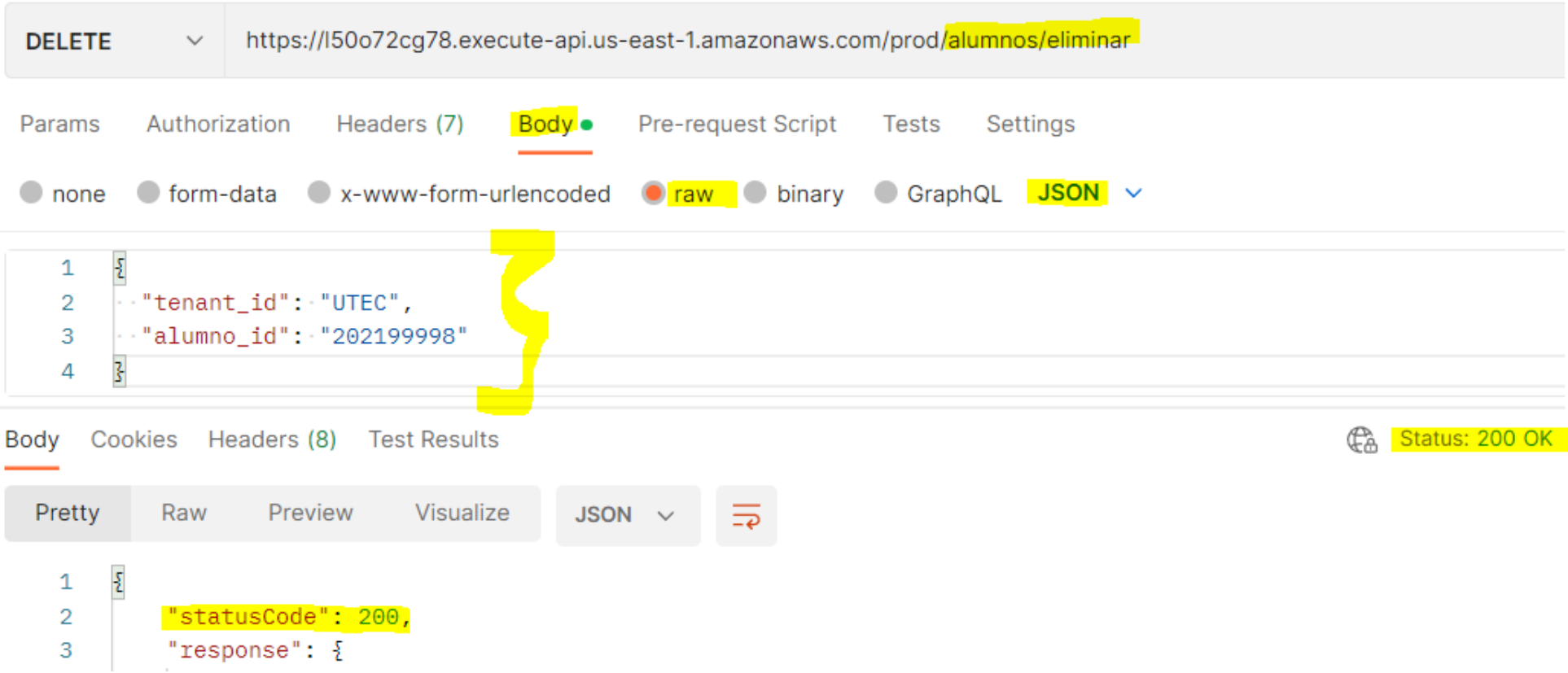
Ejercicio 6:
Publicar microservicio como API Rest CRUD con servicio Api Gateway

- **Paso 12:** Probar la api ModificarAlumno en postman



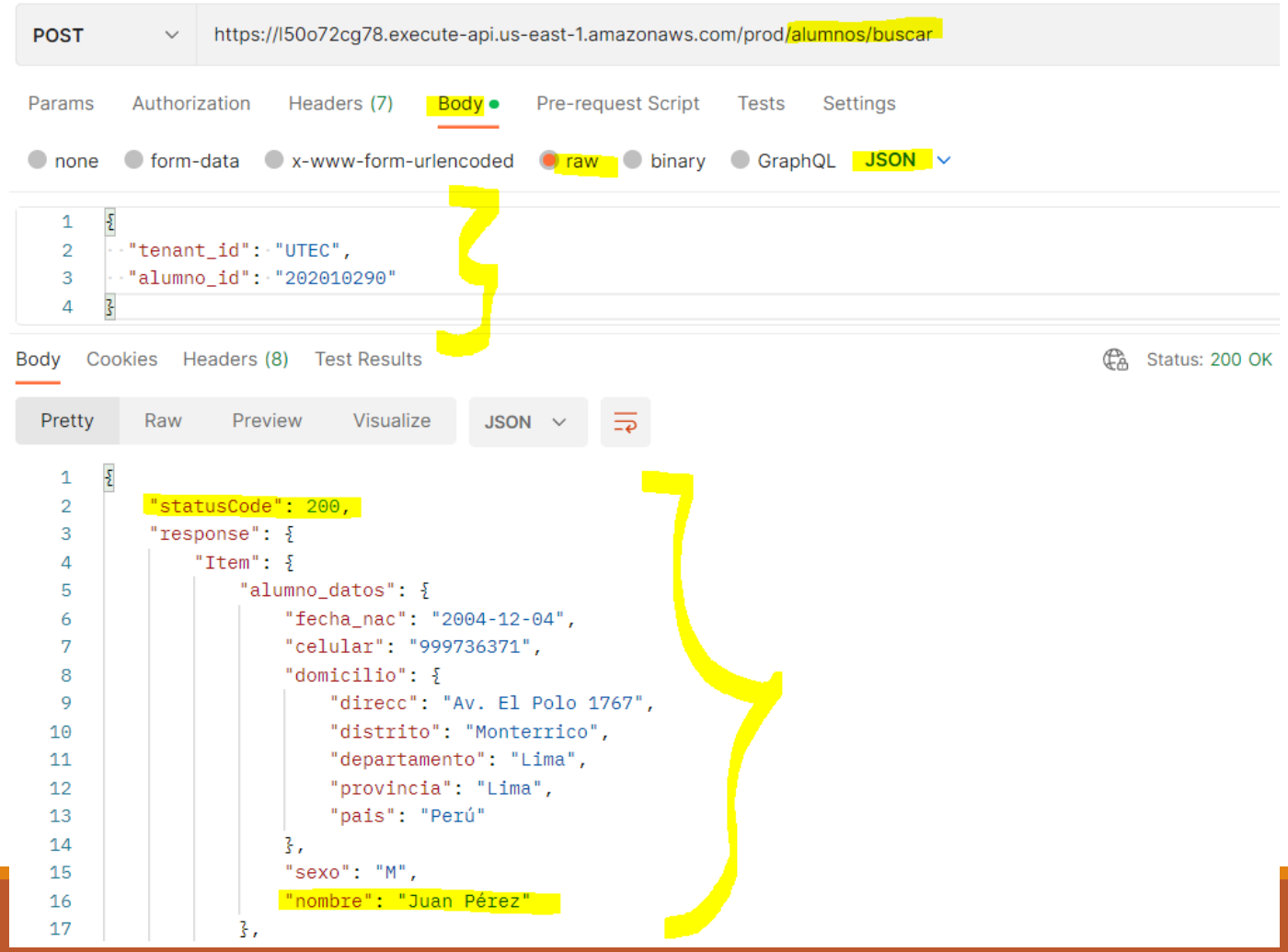
Ejercicio 6:
Publicar microservicio como API Rest CRUD con servicio Api Gateway

- **Paso 13:** Probar la api EliminarAlumno en postman



Ejercicio 6:
Publicar microservicio como API Rest CRUD con servicio Api Gateway

- **Paso 14:** Probar la api BuscarAlumno en postman



Contenido

Microservicio Multi-tenancy

1. Objetivo del taller 2
2. Ejercicio 1: Crear una función Lambda - ListarAlumnos
3. Ejercicio 2: Crear una función Lambda - CrearAlumno
4. Ejercicio 3: Crear una función Lambda - ModificarAlumno
5. Ejercicio 4: Crear una función Lambda - BuscarAlumno
6. Ejercicio 5: Crear una función Lambda - EliminarAlumno
7. Ejercicio 6: Publicar microservicio como API Rest CRUD con servicio Api Gateway
8. **Ejercicio 7: Ejercicio propuesto**
9. Cierre

Ejercicio 7:
Ejercicio propuesto para la casa

- Investigue y cree una función lambda “buscar2” y habilite su acceso con Método GET en Api Gateway que reciba los parámetros como cadena de consulta de URL. Ejemplo:

  https://c6ny37pbaj.execute-api.us-east-1.amazonaws.com/prod/alumnos/buscar2***?tenant_id=UTEC&alumno_id=202077799***

- Muestre las evidencias de postman en el padlet indicado por el docente.

Contenido

Microservicio Multi-tenancy

1. Objetivo del taller 2
2. Ejercicio 1: Crear una función Lambda - ListarAlumnos
3. Ejercicio 2: Crear una función Lambda - CrearAlumno
4. Ejercicio 3: Crear una función Lambda - ModificarAlumno
5. Ejercicio 4: Crear una función Lambda - BuscarAlumno
6. Ejercicio 5: Crear una función Lambda - EliminarAlumno
7. Ejercicio 6: Publicar microservicio como API Rest CRUD con servicio Api Gateway
8. Ejercicio 7: Ejercicio propuesto
9. **Cierre**

Cierre:
Microservicio Multi-tenancy - Qué aprendimos?

- Diseño e implementación de un Microservicio Multi-tenancy con servicio Lambda (Serverless) y Api Gateway (Serverless)

Gracias

Elaborado por docente: Geraldo Colchado